



Building a College Accountability Engine with Claude

The product I would actually build

After looking at current productivity systems, behavior-change research, Claude's current developer stack, Purdue's LMS environment, and the APIs available for health/productivity data, I would change the framing of your idea slightly.

Do not build a giant habit tracker with AI commentary. Build a personal closed-loop operating system.

The loop should be:

Observe → Plan → Execute → Detect deviation → Intervene → Reflect → Learn → Update the next plan.

That distinction matters. Exist already combines automatic and manual personal data to surface behavioral relationships; Sunsama emphasizes guided daily planning and timeboxing; Reclaim automatically schedules flexible habits around calendar constraints; and Beeminder adds real consequences to commitments. Your opportunity is to combine the strongest parts of these ideas with academic context, physiological context, passive telemetry, and an AI reasoning layer. ¹

The biggest mistake would be optimizing for **how much data you can collect**. Optimize for **how many useful decisions the system can make with the least effort from you**.

A useful rule for this project is:

Every data point should either change a decision, trigger an intervention, improve a prediction, or be removed.

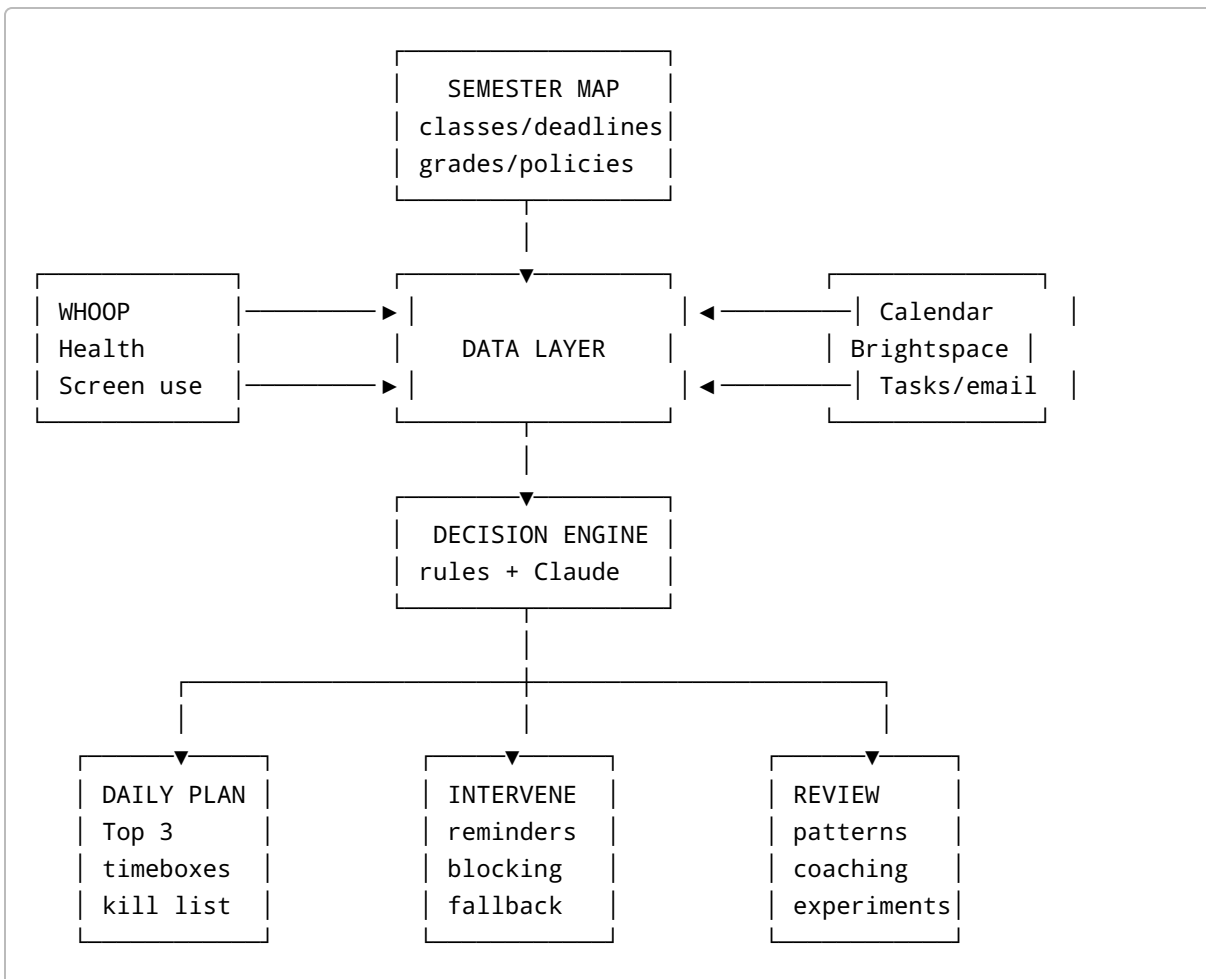
That means I would absolutely collect WHOOP, calendars, deadlines, screen usage, morning intentions, and evening reflections. But I would not start collecting Spotify history, every heart-rate sample, every GPS coordinate, every browser tab, spending transactions, weather, and thirty subjective metrics simply because APIs exist. More inputs create more maintenance burden, more privacy exposure, more spurious correlations, and more context for Claude to process.

Your concept also aligns well with behavioral research. Implementation intentions—specific "if X occurs, then I will do Y" plans—have repeatedly improved goal attainment, and a major meta-analysis covering 94 independent tests found a medium-to-large overall effect on goal achievement. ² Continuous self-

reporting, however, creates burden and adherence problems, which argues for a system where passive data is automatic and manual check-ins are extremely short. ³

So the product I would build for you is something like this:

CollegeOS



The important thing is that **Claude is not the database** and **Claude is not the scheduler**. Claude is the reasoning layer sitting on top of reliable structured data.

The feature set I would build beyond your original ideas

Your morning tracker, top three tasks, nightly journal, kill list, WHOOOP integration, screen-time integration, syllabus ingestion, and multiple coaching perspectives are strong starting concepts. I would retain them, but I would significantly expand and restructure them.

The semester intelligence layer

This may ultimately be more valuable than your habit tracker.

At Purdue, Brightspace is the supported LMS and can contain syllabi, grades, announcements, course materials, and activities. 4

The system should create a **Semester Map** for every course:

Data	Example
Course	BIOL 23000
Professor	Name/contact
Class meetings	MWF 10:30
Office hours	Tue 2-4
Exams	Sep 18, Oct 23, Dec 12
Assignments	every deadline
Assignment weight	10%, 15%, etc.
Late policy	-10%/day
Attendance policy	three free absences
Grade boundaries	A $\geq$ 93
Required readings	chapters/papers
Major projects	milestone dates
Course difficulty estimate	user/system
Current grade	optional
Target grade	A
Confidence	1-5
Current risk	calculated

From that, build a **Deadline Radar**, not merely reminders.

A deadline ten days away should not produce one alert ten days later. The system should work backwards:

Biomedical instrumentation report

Due: October 18

Oct 8 Understand requirements

Oct 10 Find sources

Oct 12 Outline

Oct 14 First draft

Oct 16 Revision

Oct 17 Final check
Oct 18 Submission

One reason this is worthwhile is that objective learning-analytics research has found meaningful relationships between procrastination behavior and academic performance using actual assignment-submission records, rather than relying only on students' memories of whether they procrastinated. 5

The academic risk engine

This is one of the strongest features I think you're currently missing.

Every class receives a continuously changing risk score.

Not something simplistic like:

CHEM = 7/10 risk.

Instead:

BME 301 — HIGH ATTENTION

Why:

- Exam worth 22% in 9 days
- You rate understanding 2.8/5
- Only 1 of 4 planned study sessions completed
- Last three study blocks started ~1.4 days later than planned
- Current projected grade is near your minimum target
- Thursday and Friday are already calendar-heavy

Recommendation:

Move one BME block to Tuesday 6:30 PM.

Do retrieval practice, not notes review.

Target: Chapters 4-5 problems.

The score could combine:

```
risk =  
deadline proximity  
× grade weight  
× estimated difficulty  
× knowledge gap  
× unfinished work ratio
```

- × schedule congestion
- × historical procrastination tendency

Claude can explain the score, but **normal code should calculate it.**

The capacity-aware planning system

WHOOOP should not simply appear on a dashboard as:

Recovery: 43%.

WHOOOP's API can expose recovery information including HRV and resting heart rate, sleep measurements and stages, cycle/strain information, and workouts. It uses OAuth 2 and supports webhooks. ⁶

Use that information to alter the plan.

For example:

```
Normal day:
Target workload = 100%

Poor sleep/recovery:
Target workload = 75%
Protect = mandatory deadlines + one high-value study block
Remove = low-value admin work
Add = earlier wind-down
```

But there is an important anti-confirmation-bias rule here:

Never let WHOOOP become an excuse generator.

The engine should use physiology primarily to adjust *how* and *when* you work, not continuously tell you not to work because a wearable gave you a yellow number.

I would therefore implement three daily workload levels:

Floor — minimum acceptable day.

Target — realistic successful day.

Stretch — additional tasks only when capacity exists.

That is much better than a giant daily task list where anything less than 100% feels like failure.

Task-duration calibration

This is a feature almost nobody thinks to build, and I think it could become extremely valuable.

Whenever you plan meaningful work, predict how long it will take:

Physics problem set: 75 minutes.

Record actual:

132 minutes.

Over time your engine learns:

Your estimate calibration

Coding assignments	×1.31
Lab reports	×1.68
Problem sets	×1.14
Reading	×0.92
Administrative tasks	×1.05

Then the scheduler stops believing your optimistic estimates.

Sunsama explicitly encourages users to estimate task duration before timeboxing work. ⁷ Your system can go one step further by learning the personal correction factor.

A much better kill list

Your kill list should absolutely exist, but I would not make it simply:

- ✗ Instagram
- ✗ Porn
- ✗ Staying up late

That creates very little information.

Create a **Kill Loop**:

```
Behavior
↓
Trigger
↓
```

```
Urge
↓
Action
↓
Immediate reward
↓
Long-term cost
↓
Replacement behavior
```

A kill-list event could be captured in five seconds:

```
Instagram relapse
Time: 4:42 PM
Trigger: difficult homework
Duration: automatically detected
Mood before: frustrated
```

Then Claude eventually discovers:

"Most social-media failures aren't random. They cluster immediately after difficult study tasks and during transitions between classes."

That insight is much more actionable than:

"You spent 74 minutes on Instagram."

For each kill habit, create an implementation intention:

```
IF I reach for Instagram while stuck on schoolwork,
THEN I write down the question I'm stuck on and work on it
for five more minutes before deciding whether to take a break.
```

This structure follows evidence around if-then implementation intentions. 8

Friction logging

Another feature I strongly recommend:

When something fails, ask **why in one tap**.

```
Why wasn't MIT #1 completed?
```

- underestimated duration
- didn't know next step
- distracted
- tired
- schedule changed
- avoided uncomfortable task
- higher priority appeared
- other

This produces something extremely useful:

a database of why Mehdi fails.

After a semester you might learn:

Top reasons important work was missed

32% underestimated duration
23% unclear next action
17% phone distraction
14% unexpected schedule conflicts
8% low energy
6% other

Now your accountability engine becomes diagnostic rather than judgmental.

The comeback metric

Do not optimize primarily for streaks.

Track:

time-to-recovery after failure.

Example:

Killed habit failed Tuesday.
Old pattern:
failure → four-day slide.

New pattern:
failure → normal behavior next morning.

That is real improvement even though your streak counter says both cases are failures.

Create:

Bounce-back score

How rapidly do you resume your intended behavior after breaking it?

I think this is psychologically much healthier and more useful than worshipping perfect streaks.

Minimum viable day

Your engine needs a disaster mode.

College guarantees occasional terrible days.

Instead of the system watching you fall hopelessly behind, it should detect:

```
poor sleep
+ behind schedule
+ missed morning check-in
+ multiple deadlines
```

and switch into:

Recovery Mode

Example:

TODAY'S MINIMUM VIABLE DAY

Must:

1. Submit BME homework
2. Attend PHYS lab
3. 30 min exam preparation

Protect:

- sleep by 11:45
- no social media during study block

Everything else rolls forward automatically.

This prevents all-or-nothing failure cascades.

Proof-of-work accountability

For particularly important tasks, let tasks require evidence.

Examples:

```
"Finish lab report"
→ attach submitted confirmation.

"Study organic chemistry"
→ 25 completed practice questions.

"Work on coding project"
→ Git commit.

"Read research paper"
→ 3-sentence summary.

"Go to gym"
→ WHOOP workout.
```

Now your accountability engine doesn't have to trust your ability to rationalize whether something was "basically done."

Commitment escalation

I would borrow one concept from Beeminder: optional commitments with real consequences. Beeminder explicitly uses commitment contracts and monetary consequences as an anti-akrasia mechanism. ⁹

Your version should be gradual.

```
Level 0 – reminder
Level 1 – stronger notification
Level 2 – distraction apps blocked
Level 3 – accountability partner notified
Level 4 – predetermined consequence
```

Do not begin with punishment.

Make consequences available only for very specific behaviors that you have repeatedly failed despite weaker interventions.

Weekly experiments instead of endless correlations

You will eventually have enormous amounts of data.

A dangerous temptation will be producing statements like:

"On days you drink coffee after 2 PM your productivity is 18% lower."

That does **not** mean the coffee caused it.

Your system should label observational relationships as hypotheses, then deliberately test them.

Example:

OBSERVATION

Late caffeine correlates with later sleep.

EXPERIMENT

For 7 days:

No caffeine after 1:00 PM.

OUTCOMES

Sleep onset

Sleep duration

WHOOP recovery

Morning energy

MIT completion

This turns the engine into a miniature N-of-1 experimentation platform rather than a correlation hallucination machine.

Planning versus execution gap

Measure:

planned deep work: 180 min

actual deep work: 94 min

planned MITs: 3

completed MITs: 2

planned start time: 4:00

actual start time: 5:07

Then separately measure:

planning quality and **execution quality**.

You may discover that your problem isn't motivation at all. It might be chronic overplanning.

Focus-session intelligence

Every meaningful study block gets:

```
planned duration
actual duration
task
course
location
start delay
interruptions
phone usage
subjective focus
objective output
```

Eventually the engine could learn:

"Your most reliable 90-minute blocks occur 10 AM–1 PM, especially in the library. Evening sessions are much more likely to start late."

That can actually change scheduling decisions.

A daily prediction

Each morning you should predict:

```
Probability I complete all MITs: 70%
Expected energy tonight: 5/10
Hardest task: BME problem set
Most likely failure mode: phone after class
```

At night, score the prediction.

This trains self-awareness and gives the engine calibration data.

Decision journal

For important choices:

Decision:
Skip tonight's study block and study tomorrow morning.

Why:
I believe I'll be sharper then.

Prediction:
80% chance I complete it before class.

Later:

Outcome:
Didn't happen.

After enough entries, Claude can identify systematic decision errors.

Grade scenario planning

Once syllabus weights and current grades are known:

Current projected course grade: 88.4

To reach 90:

Exam 3: ≥ 92

Project: ≥ 91

If Exam 3 = 85:

Final exam required for A = 96

If Project = 95:

Final exam required for A = 91

This lets your planner allocate effort according to **academic return on time**, rather than anxiety.

Office-hours intelligence

Automatically capture professor/TA office hours from syllabi and surface them contextually.

Example:

"You've marked this topic confusing for three consecutive sessions. Professor office hours are tomorrow 2–4 PM."

That is a much better intervention than Claude explaining the concept for the tenth time.

Semester retrospectives

At semester end:

What predicted good academic days?
What predicted bad ones?
Which habits mattered?
What was noise?
Which class planning assumptions were wrong?
Which kill behaviors improved?
What schedule worked?
What should next semester's system change?

Then carry only the highest-confidence lessons forward.

That gives your accountability engine actual long-term memory.

APIs and data sources worth connecting

I would rank integrations by **decision value divided by implementation complexity**, not by how exciting they sound.

Highest priority integrations

Source	What you gain	Feasibility	My priority
WHOOP	sleep, recovery, HRV, strain, workouts	Excellent	Immediate
Calendar	classes, meetings, workload	Excellent	Immediate
Syllabus PDFs	deadlines, grading, policies	Excellent	Immediate
Brightspace calendar feed	academic dates/tasks	Good	Immediate
RescueTime	computer apps/sites/productivity	Excellent	Immediate
Morning/night forms	subjective context	Excellent	Immediate
Todo/task database	planned/completed work	Excellent	High
Screen Time	phone distraction	Technically harder on iOS	High

Source	What you gain	Feasibility	My priority
Apple Health	broader health/activity	Native iOS required	Medium-high

WHOOOP is particularly attractive because its current developer API exposes recovery, sleep, cycles, workouts and associated physiological measurements, with OAuth 2 and webhook support. ¹⁰

For desktop productivity, **RescueTime is probably more useful than trying to build computer telemetry yourself.** Its current analytics API can return activity history grouped by application, category, productivity level, or document and can operate at minute/hour/day/week/month resolution; it also exposes daily summaries and Focus Session history. ¹¹

That would give you something like:

```
Laptop:
VS Code      118m
Brightspace   42m
Chrome research 67m
YouTube      29m
Reddit       22m
```

combined with your phone behavior.

The iPhone Screen Time situation

This is feasible but considerably more complicated than WHOOOP.

Apple provides Screen Time technology through **FamilyControls, DeviceActivity, and ManagedSettings.** Since iOS 16, individual users—not just parents managing children—can authorize Screen Time functionality for themselves. ¹²

That means a native iOS companion could potentially:

```
Monitor selected applications/categories
Detect usage thresholds
Display usage reports
Shield/block distracting applications
```

But this is not equivalent to a convenient web REST endpoint where your backend requests:

```
GET /mehdi/screentime/today
```

It requires native Apple frameworks and privacy-preserving architecture. Apple developer discussions also illustrate that developers commonly implement usage displays inside `DeviceActivityReport` extensions, making this integration materially more complicated than a normal OAuth API. ¹³

Therefore:

Do not make iPhone Screen Time a requirement for MVP launch.

Make it the first major native feature after the core loop works.

Apple Health

HealthKit is Apple's central repository for health and fitness information on iPhone and Apple Watch, with explicit user permissions for the data categories an application reads. ¹⁴

Potential useful signals:

```
steps
exercise
sleep
heart rate
walking/running activity
workouts
```

Because you already use WHOOP, there will be overlap. I would initially use WHOOP as the physiological source of truth and HealthKit only for information WHOOP doesn't provide well enough.

Purdue Brightspace

This deserves special attention because you are at Purdue.

Purdue currently uses Brightspace as its supported LMS, including course content, activities, resources and grades. ⁴

Brightspace has a substantial API surface, including scopes for calendar, content, dropbox/assignments, grades, news, quizzes and other academic information. ¹⁵

But there is an obstacle.

D2L's recommended OAuth flow requires an OAuth application to be registered through Brightspace's **Manage Extensibility** administration interface. ¹⁶

As an ordinary student, you should therefore **not assume Purdue will let your personal application directly obtain unrestricted Brightspace API access.**

Fortunately, there is a much easier route.

Brightspace supports **iCal feeds** that can subscribe external calendars to course events and tasks. D2L currently documents the ability to select a course calendar or "All Calendars and Tasks," obtain a feed URL, or download an ICS file. ¹⁷

So my academic ingestion hierarchy would be:

```
Best:
Brightspace API
(if Purdue eventually permits your integration)

↓

Very good:
Brightspace iCal feed
+
syllabus upload

↓

Fallback:
syllabus upload
+
calendar
+
manual quick-add
```

For your MVP, **Brightspace iCal + syllabus ingestion is enough.**

Calendar and university email

Microsoft Graph can access Outlook calendar and mail in Microsoft 365 and supports calendar events, mail, and change-tracking capabilities. ¹⁸

That opens a useful future capability:

```
Professor email:
"Problem set 4 has been moved to Friday."

↓

CollegeOS detects potential deadline change.

↓
```

```
"Possible deadline update:  
BME 301 PS4 → Friday.  
Confirm?"
```

The important word is **confirm**.

Never let an LLM silently alter an academic deadline from an email without validation.

Todoist or Notion

Both are viable.

Todoist now provides its own API plus tooling aimed at integrations and agents. ¹⁹

Notion's API can read/create/update workspace content using authenticated connections. ²⁰

But I would strongly resist making Notion the internal database.

Use Postgres as the real system of record.

Notion can be a **view/export surface** later.

Research and reading

Because you're in biomedical engineering, Zotero is surprisingly interesting.

Zotero's current Web API exposes library items, collections, tags and even full-text content of indexed attachments; its desktop application also exposes a local API. ²¹

A future research agent could see:

```
Papers assigned this week  
Papers saved but unread  
Research reading completed  
Paper summaries  
Citation library
```

Readwise also exposes an API for reading and updating highlights and now explicitly supports agent-oriented access. ²²

Neither belongs in MVP, but both could become useful for upper-level coursework and research.

Other possible signals

I would divide remaining integrations this way:

Probably useful later: GitHub activity for coding classes, Zotero, Readwise, location-based study context, expense data if financial discipline becomes one of your goals, and weather only if it proves predictive of routines.

Probably noise initially: Spotify, social media engagement metrics beyond distraction time, individual heartbeat streams, detailed GPS traces, every browser URL, continuous mood sampling.

The objective should remain:

minimal manual input + high-quality passive context + strong interventions.

The agent system should not actually be a swarm of agents

You mentioned a coach, motivator, psychologist, etc.

Conceptually I like multiple perspectives.

Architecturally I would **not** start by running eight autonomous Claude agents every night.

That increases cost, latency, duplicated reasoning, inconsistent recommendations, and the probability that different agents tell you contradictory things.

Instead I would create one analysis pipeline with multiple **lenses**.

The lenses I would use

Executive Coach

Looks at whether actions match stated priorities.

"You keep saying your most important goal is academics, but only 34% of discretionary time this week was allocated to coursework."

Academic Strategist

Looks at deadlines, grades, workload and course risk.

"BME requires attention before CHEM because its exam is closer and represents more of the grade."

Behavior Analyst

Looks for triggers, repeated behavioral patterns and environment effects.

"Phone failures happen disproportionately after difficult study blocks."

I would use this instead of a "psychologist" persona. The system should help you reflect on behavior without pretending it is performing psychological diagnosis.

Skeptic / Adversarial Reviewer

This is important because you explicitly told me not to confirmation-bias you.

Its job is:

What story is Mehdi telling himself?
What evidence contradicts it?
What alternative explanation exists?
Is this correlation being treated as causation?
Is the system measuring the wrong thing?

Example:

"You're blaming low recovery for missed workouts, but three of the four misses occurred on normal-recovery days. Scheduling appears more explanatory."

This may become one of your best agents.

Motivator

This should be used sparingly.

Not:

"You're amazing! Keep pushing!"

Instead:

"Yesterday collapsed after 4 PM. You still recovered and completed the assignment before midnight. Today's goal is not perfection; it is starting the BME block at 3:30."

Specific evidence beats generic hype.

Systems Engineer

Asks:

"How can we redesign the environment so this no longer requires willpower?"

For example:

Repeated problem:
Instagram after returning from class.

System intervention:
Automatically activate blocker from
3:30–5:30 on weekdays.

Recovery Coach

Activates only when you're clearly overwhelmed.

Its goal is avoiding cascades.

Longitudinal Analyst

Only runs weekly/monthly.

It finds multi-week trends rather than responding emotionally to one bad day.

How one Claude call can produce all of them

Send Claude structured information like:

```
{
  "date": "2026-09-22",
  "morning": {
    "energy": 6,
    "mood": 7,
    "top_tasks": [
      "Finish BME lab report",
      "Study PHYS 241",
      "Gym"
    ]
  },
  "physiology": {
    "sleep_hours": 6.8,
    "whoop_recovery": 58
  },
  "execution": {
    "tasks_completed": 2,
    "planned_focus_minutes": 180,
```

```

    "actual_focus_minutes": 123,
    "social_media_minutes": 71
  },
  "kill_list": {
    "instagram": false,
    "late_bedtime": true,
    "youtube_in_bed": false
  },
  "journal": "...
}

```

and request one structured response:

```

{
  "executive_coach": "...",
  "academic_strategist": "...",
  "behavior_analyst": "...",
  "skeptic": "...",
  "systems_engineer": "...",
  "tomorrow_intervention": "...",
  "confidence": {
    "claim_1": 0.78
  }
}

```

You still experience six "agents."

You pay for **one inference**.

Do not send your entire life history every night

This is a major architecture choice.

Claude models currently support large context windows—Sonnet 5 is listed with a one-million-token context window—but having room for enormous histories doesn't mean continually resending them is intelligent or economical. ²³

Store:

```

Raw event history
↓
Daily summaries
↓
7-day rolling summary
↓

```

30-day pattern summary
↓
Semester-level durable lessons

Nightly Claude sees:

today
+ relevant previous 7 days
+ durable profile
+ active goals
+ upcoming academic obligations

Weekly Claude sees more.

Monthly Claude sees aggregated history.

That is far cleaner.

The architecture I would use with Claude

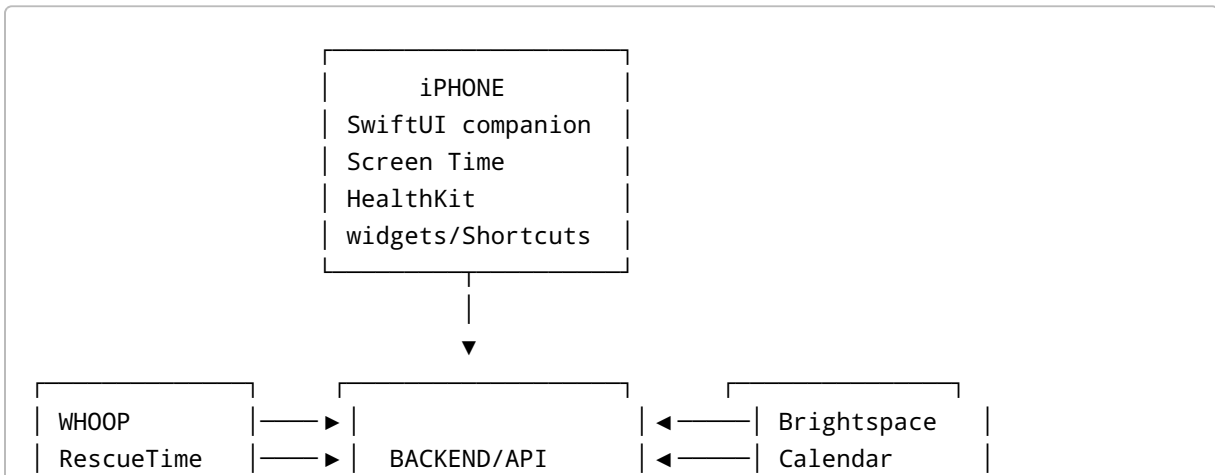
Use Claude Code to build it, not to be the database

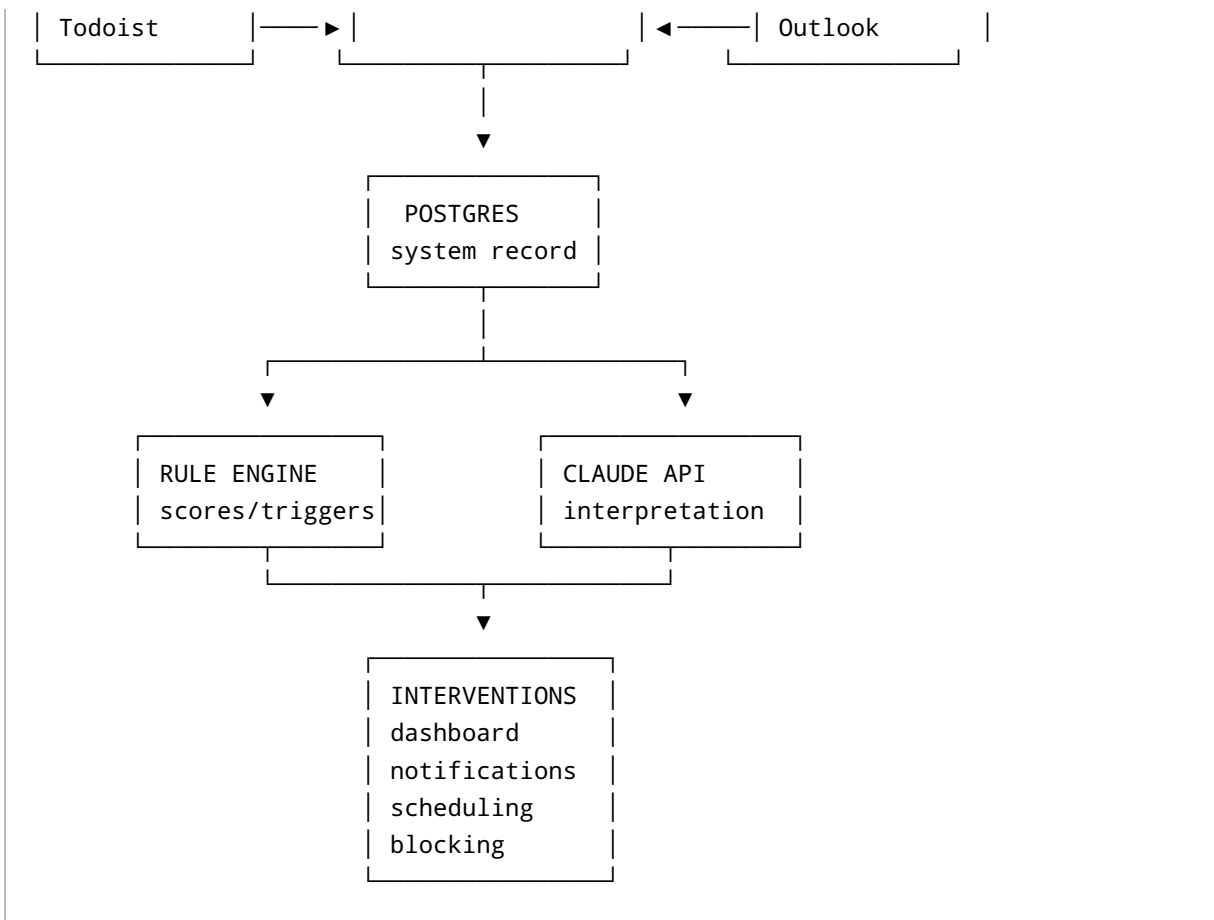
Your \$100/month Claude subscription is currently the **Max 5x plan**, which Anthropic describes as providing five times Pro's session capacity and access to Claude Code. ²⁴

Claude Code is very well suited to actually building this because it supports MCP connections, subagents, hooks, project instructions, plugins and custom automation. ²⁵

I would have Claude Code create the application, tests, migrations, integration clients and prompts.

I would use this architecture:





The technology stack

My recommendation for a personal project:

Layer	Choice
Web UI	Next.js + TypeScript
Styling	Tailwind
Database	PostgreSQL
Hosted backend	Supabase or similar
Authentication	Supabase Auth
Scheduled ingestion	backend cron/worker
Charts	lightweight charting library
LLM	Anthropic Messages API
Mobile later	Native SwiftUI

Layer	Choice
Screen Time	FamilyControls + DeviceActivity + ManagedSettings
Health	HealthKit
Development agent	Claude Code
API abstraction	ordinary TypeScript clients first, MCP where useful

I would specifically **not use React Native as the first solution for the Screen Time component**.

Because Apple's Screen Time architecture relies on native frameworks and app extensions, a thin native SwiftUI companion will give you much less friction than trying to hide all of those capabilities behind a cross-platform abstraction. Apple's documented Screen Time frameworks support precisely the kind of individual authorization and device-activity control this feature requires. ¹²

The database model

You do not need an insane schema.

The foundational tables could be:

```

users

courses
course_assignments
course_exams
course_policies

tasks
task_sessions
task_predictions

daily_checkins
daily_reviews

habits
habit_events

kill_habits
kill_events

health_daily
screen_daily
app_usage

calendar_events

```

```
journal_entries  
  
experiments  
experiment_measurements  
  
agent_reports  
  
insights  
interventions
```

Most telemetry should ultimately be representable as generic events:

```
{  
  "timestamp": "...",  
  "source": "whoop",  
  "type": "sleep",  
  "metric": "sleep_duration",  
  "value": 6.82,  
  "unit": "hours"  
}
```

That gives you flexibility to add APIs later without rebuilding the entire database.

Keep calculations deterministic

Do this in code:

```
streaks  
completion rates  
screen-time totals  
task lateness  
assignment risk  
grade calculations  
time estimates  
sleep averages  
rolling averages  
trend slopes  
calendar free time  
study minutes  
correlations
```

Use Claude for:

```
interpretation
prioritization
reflection
hypothesis generation
coaching
challenging rationalizations
explaining patterns
generating interventions
```

This will drastically reduce hallucination risk.

Use Claude's structured output rather than free-form essays

Do not have the backend ask:

"Tell Mehdi how his day went."

Have it produce typed objects.

```
interface DailyAnalysis {
  headline: string;
  wins: EvidenceClaim[];
  failures: EvidenceClaim[];
  patterns: EvidenceClaim[];
  academicRisk: RiskItem[];
  tomorrowChanges: Intervention[];
  coach: string;
  skeptic: string;
}
```

Then your interface decides what is shown.

Privacy architecture matters more than usual here

This system will contain journals, behavioral vulnerabilities, academic records and potentially physiological information.

I would therefore keep raw data in your database and only send **the minimum required subset** into Claude.

Example:

Do not send:

```
every heart-rate reading for the past week
```

Send:

```
sleep = 6.3 h  
recovery = 42  
HRV deviation = -11% vs 30d baseline
```

Anthropic's current commercial API documentation says ordinary API conversation content is not retained by default, with an exception for designated Covered Models such as Fable 5/Mythos 5, which require 30-day retention; Anthropic also offers formal zero-data-retention arrangements for qualifying organizations. Consumer Claude Free/Pro/Max surfaces are not covered by those ZDR arrangements. ²⁶

For this project I would consequently route sensitive journaling primarily through **Haiku/Sonnet API calls**, store the canonical history yourself, and avoid unnecessarily placing your whole personal database into Claude projects.

The end-to-end experience should be almost frictionless

Convenience is arguably the most important product requirement you identified.

I would design the experience around one principle:

The system asks you only for things it cannot infer automatically.

Initial semester setup

At the beginning of a semester:

```
Welcome back.
```

```
Connect:
```

```
✓ Calendar  
✓ WHOOP  
✓ RescueTime
```

```
Optional:
```

```
○ Todoist  
○ Outlook  
○ HealthKit
```

```
Courses detected/imported:
```

```
BME ...
```

PHYS ...
...

Upload syllabus for each class.

Claude parses each syllabus into:

course information
deadlines
exam dates
grading weights
late policies
attendance rules
office hours
required texts

Then show a confirmation page:

I found 41 dated items.

Highest importance:
Sep 17 – Exam 1
Oct 3 – Lab report
Oct 22 – Exam 2

Review uncertain dates:
"Final project due during finals week"
→ needs confirmation

Do **not** silently trust LLM extraction for academic deadlines.

Sunday weekly planning

The system already knows:

calendar availability
deadlines
course risk
unfinished tasks
habit targets
recent task-duration errors

It presents:

THIS WEEK

Academic load: HIGH

Highest risk:

1. BME exam – 8 days
2. PHYS problem set – 3 days
3. CHEM lab – 5 days

Recommended deep-work allocation:

BME 5h

PHYS 3h

CHEM 2h

Suggested focus blocks:

Mon 4:30–6

Tue 6–7:30

Wed 3–5

Thu 5–6:30

Sat 11–1

You adjust once.

The engine carries the plan forward.

Morning check-in

This should take roughly a minute of interaction, not ten minutes of journaling.

Example:

Good morning.

WHOOOP

Sleep: 7h 19m

Recovery: 72

Calendar:

Class 9:30

Lab 1:30

Dinner 6:00

Yesterday's unfinished:

PHYS Q7–12

How do you feel?

Energy: [1 2 3 4 5 6 7 8 9 10]

Mood: [1 2 3 4 5 6 7 8 9 10]

Today's Top 3

1. _____

2. _____

3. _____

Most likely thing to derail you:

[phone] [fatigue] [avoidance] [schedule] [other]

Start day

Better yet, the engine should pre-fill your top priorities:

Suggested:

1. Finish PHYS assignment

2. 60m BME exam prep

3. Gym

Accept all

So most mornings require only corrections.

Convert intentions into execution plans

Do not stop at:

"Study BME."

Turn it into:

BME exam preparation

4:30–5:45 PM

WALC library

Chapter 5 practice questions

Target: questions 1–15

Phone block: ON

That is much closer to an implementation intention than a task list and is consistent with research showing the value of specifying when, where and how goal-directed behavior will occur. ²⁷

During the day

You should barely interact with CollegeOS.

Notifications should be **exception based**, not constant.

Bad:

```
11:00 Remember your goals!  
12:00 Keep going!  
1:00 Stay motivated!  
2:00 You got this!
```

Good:

```
4:22 PM  
  
Your BME block begins in 8 min.  
You've used 41 min of social media today.  
  
Start Focus Mode?  
[Start] [Move block]
```

Or:

```
5:03 PM  
  
BME block hasn't started.  
  
Why?  
[Forgot] [Avoiding] [Schedule changed] [Start now]
```

One tap generates useful behavioral data.

Focus mode

Eventually, this should become one of the killer features of your native iPhone companion.

```
START BME FOCUS — 75 MIN  
  
Block:  
Instagram  
TikTok
```


Reddit
YouTube
X

Allowed:
Messages
Phone
Calculator
Safari

Apple's Screen Time APIs were explicitly expanded to support individual authorization for self-control use cases rather than only parent-child controls. ²⁸

When the session ends:

Did you accomplish the target?

[Yes] [Partly] [No]

Actual progress:
15/15 problems

Focus:
1 2 3 4 5

Done.

Five seconds.

Night review

Again: keep it short.

Most of the review should already be filled in.

TODAY

MITs	2 / 3
Deep work	2h 18m
Screen time	3h 12m
Distracting	58m
Workout	✓
Kill list	2 / 3 successful
Sleep target	pending

What are you proud of?
[voice/text]

What went wrong?
[voice/text]

Anything important about today?
[voice/text]

Submit

Voice input may be extremely valuable here because speaking for sixty seconds is much easier than writing a long journal every evening.

Nightly engine

Now code gathers:

```
morning prediction
tasks
actual task sessions
calendar
screen behavior
health
kill events
night journal
academic deadlines
```

Then Claude generates a concise report.

Example:

Main win: You completed both high-priority academic tasks despite starting the day below your target energy.

Main failure: The gym task was unrealistic once the PHYS assignment overran by 46 minutes.

Behavior pattern: Your largest distraction period occurred during the transition after lab, not during scheduled work.

Skeptic: You described today as "unproductive," but you completed both academically critical MITs. Your emotional rating and objective output disagree.

Tomorrow change: Do not schedule three high-effort tasks after 4 PM. Move exercise before dinner.

Kill-list intervention: Trigger Focus Mode automatically during tomorrow's post-class transition.

That's useful.

Five paragraphs of motivational filler is not.

Weekly report

Weekly is where Claude should do deeper reasoning.

I would structure it around:

OUTCOMES

What actually happened?

PLAN ACCURACY

Were estimates realistic?

ACADEMICS

Which courses are drifting?

BEHAVIOR

Which repeated triggers appeared?

HEALTH

Any meaningful relationship with execution?

KILL LIST

Triggers + recovery speed

SYSTEM FAILURE

What about CollegeOS itself isn't working?

EXPERIMENT

One thing to change next week.

The **system failure** section is critical.

CollegeOS should occasionally conclude:

"Morning question #7 has produced no actionable insight for four weeks. Remove it."

The accountability engine should optimize itself.

Credit usage, Claude strategy, and the MVP I would build

Your \$100 Claude plan does not equal \$100 of API

This is the most important billing fact to understand.

Your \$100/month Claude plan corresponds to Max 5x, which gives greater interactive Claude usage and Claude Code access. ²⁴

Anthropic explicitly states that paid Claude subscriptions and Claude API/Console access are separate products; subscribing to Max does **not** automatically give you ordinary API credits for an application. ²⁹

So separate two budgets:

```
$100 Max subscription
→ build CollegeOS with Claude Code
→ coding
→ debugging
→ architecture
→ refactors

Claude API account
→ CollegeOS actually analyzing days automatically
→ small pay-as-you-go budget
```

There is currently some movement around Agent SDK/subscription usage. Anthropic's June 2026 help documentation says planned changes were paused and Agent SDK/ `claude -p` usage currently continues drawing from subscription limits, while its developer documentation also warns that third-party products should generally use API-key authentication rather than offering Claude subscription login. ³⁰

Because that policy surface is actively evolving, **I would not architect your personal operating system around exploiting your Max subscription for unattended backend calls.**

Use Max aggressively for Claude Code development.

Give the actual app a Claude API key and a hard monthly spend limit.

Current Claude API economics are actually favorable

As of August 11, 2026, Anthropic lists:

Model	Input / million tokens	Output / million tokens
Haiku 4.5	\$1	\$5

Model	Input / million tokens	Output / million tokens
Sonnet 5 through Aug. 31	\$2	\$10
Sonnet 5 from Sep. 1	\$3	\$15
Opus 5	\$5	\$25
Fable 5	\$10	\$50

31

I would build around:

Haiku = extraction / classification / easy daily processing

Sonnet = actual coaching / reasoning / weekly synthesis

Opus = rare semester-level or difficult analyses

Fable = unnecessary for this product

A conservative monthly cost model

Assume after Sonnet's introductory August pricing ends:

Workflow	Model	Approx. monthly
Morning planning	Haiku	~\$0.11
Nightly multi-lens analysis	Sonnet	~\$0.90
Weekly reviews	Sonnet	~\$0.26
Ten on-demand coach conversations	Sonnet	~\$0.20
Six syllabus extractions	Haiku	~\$0.12 one-time
Estimated first-month inference		~\$1.60

Those figures assume roughly:

Morning:
1.5k input
400 output

Night:
4k input

1.2k output

Weekly:

10k input

2k output

Even adding generous overhead for prompt structure, retries, larger summaries and occasional extra conversations, I would initially budget **about \$2-5/month of API usage**, not anywhere near \$100.

The system becomes expensive primarily if you repeatedly send giant histories, use top models for trivial extraction, or run many independent agents over identical context.

Prompt caching can cut repetitive context dramatically

Current Claude pricing charges Sonnet 5 cache hits at \$0.30 per million tokens after September 1 compared with \$3 per million for ordinary input—a 90% reduction for cached input. Haiku cache hits are \$0.10 per million compared with \$1 ordinary input. ³²

So cache your stable information:

```
user profile
semester goals
agent instructions
course policies
active habits
kill list definitions
```

Anthropic also provides token counting specifically so applications can estimate usage before sending calls. ³³

Weekly/monthly jobs that do not need immediate responses could eventually use Anthropic's Batch API, which currently discounts both input and output token costs by 50%. ³⁴

The MVP

I would **not** build every integration above.

The first version should answer one question:

Does this system reliably change what you do tomorrow?

The MVP therefore contains only eight systems:

MVP component	Include?
Morning check-in	Yes
Top 3 MITs	Yes
Kill list	Yes
Night journal	Yes
Course/syllabus intelligence	Yes
Calendar/deadline radar	Yes
WHOOOP	Yes
Claude nightly + weekly analysis	Yes
RescueTime	Soon
Screen Time native app	Later
HealthKit	Later
Email parsing	Later
Zotero/Readwise	Later
Accountability partner	Later
Monetary commitment	Later

MVP screens

The entire first application only needs:

```

/ today
/ courses
/ calendar
/ review
/ insights
/ settings

```

Today

Tuesday Aug 25

Sleep 7:24
Recovery 68

TOP 3

- ☐ BME problem set
- ☐ PHYS exam study
- ☐ Workout

KILL LIST

- ✓ no Instagram before 6 PM
- ✓ no YouTube in bed
- ✓ lights out by 12

UPCOMING

PHYS homework	2 days
BME quiz	4 days
CHEM exam	9 days

Start focus session

Courses

BME 301	MEDIUM
Current grade	91
Target	93
Next deadline	Thursday
PHYS 241	HIGH
Current grade	88
Target	90
Exam	9 days

Review

Daily history and Claude reports.

Insights

Only persistent patterns with evidence.

HIGH CONFIDENCE

Your planned study durations are
28% shorter than actual durations.

MEDIUM CONFIDENCE

Social media increases on days when
your first deep-work block starts late.

TESTING

You may focus better between
10 AM and 1 PM.

That confidence hierarchy is important.

Claude should not present every pattern as fact.

MVP nightly prompt

The main nightly system prompt can essentially be:

You are the analysis engine for a personal college accountability system.

Your goal is behavior change, not reassurance.

You receive structured information about:

- goals
- academic obligations
- today's intentions
- actual execution
- physiological context
- distraction behavior
- habit outcomes
- the user's reflection

Rules:

1. Prefer objective behavior over the user's self-assessment when they conflict.
2. Do not infer causality from correlation.
3. Identify alternative explanations.
4. Challenge rationalizations when evidence supports doing so.
5. Do not manufacture patterns from insufficient data.
6. Prefer changing systems/environment over recommending more motivation.
7. Recommend no more than three changes for tomorrow.
8. Distinguish observation, hypothesis, and conclusion.
9. Never diagnose psychological or medical conditions.
10. Be direct but not insulting.

Return structured JSON containing:

headline
objective_summary
wins

```
failures
planning_errors
behavior_patterns
academic_risks
skeptical_review
systems_changes
tomorrow_plan_changes
confidence_levels
```

This gives you the coach, analyst, skeptic and systems designer without wasting four separate calls.

The Claude Code build brief

This is approximately what I would give Claude Code to begin implementation:

```
Build a personal college accountability web application called CollegeOS.
```

```
Stack:
```

- Next.js
- TypeScript
- PostgreSQL
- Supabase authentication/storage
- Anthropic Messages API

```
Core entities:
```

```
User
Course
CoursePolicy
Assignment
Exam
Task
TaskSession
DailyCheckin
DailyReview
Habit
HabitEvent
KillHabit
KillEvent
HealthDaily
CalendarEvent
JournalEntry
AgentReport
Insight
Intervention
```

```
Requirements:
```

1. Morning check-in
 - energy 1-10
 - mood 1-10
 - top 3 tasks
 - predicted probability of completing top 3
 - likely derailment reason
 - kill-list commitments
2. Night review
 - automatically show completed tasks
 - actual focus time
 - kill-list results
 - journal text
 - short structured failure reason inputs
3. Courses
 - syllabus upload
 - extract course metadata, assignments, exams, grading weights, office hours, late policies
 - never save extracted deadlines until user confirms them
4. Semester dashboard
 - upcoming deadlines
 - course risk
 - academic workload
5. WHOOP integration
 - OAuth
 - daily sleep/recovery/cycle/workout ingestion
 - store normalized daily summaries
6. Brightspace
 - support user-provided iCal feed
 - normalize events into academic calendar
7. Claude
 - Haiku for extraction/classification
 - Sonnet for nightly/weekly analysis
 - typed structured JSON responses
 - store reports in Postgres
 - do calculations outside the LLM
8. Cost controls
 - log input/output tokens
 - hard configurable monthly cost ceiling
 - no LLM call if deterministic logic suffices
 - don't send full historical journals

- create rolling summaries

9. Security

- secrets server side only
- OAuth credentials encrypted
- least-privilege scopes
- journal data never exposed to frontend logs
- validate all LLM structured output

10. UX

- mobile-first
- morning workflow should require minimal interaction
- night workflow should require minimal interaction
- one-click acceptance of suggested MITs
- avoid unnecessary notifications

Start by:

1. designing database schema
2. generating migrations
3. building mock-data version
4. writing tests for deadline/risk calculations
5. implementing morning/night workflows
6. implementing syllabus parser
7. implementing WHOOP
8. implementing Claude analysis
9. implementing Brightspace iCal
10. building insight dashboard

Do not build Screen Time or HealthKit yet.

Create interfaces so native iOS data can be added later.

Claude Code is designed for exactly this sort of repository-level coding workflow and can be extended with project instructions, subagents, hooks, and MCP data/tool connections as the project becomes more complicated. ³⁵

The development order I would personally follow

The crucial sequencing is:

DATA MODEL

↓

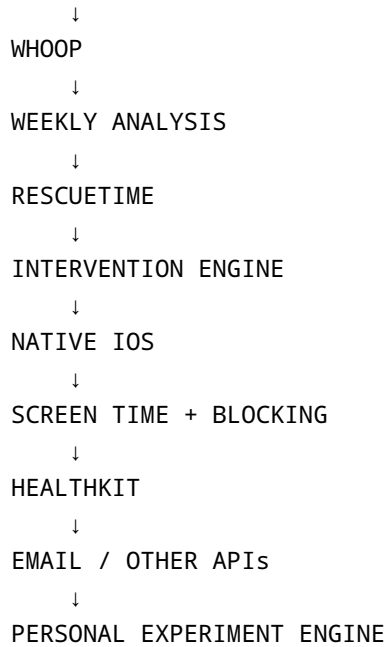
MANUAL MORNING + NIGHT

↓

COURSES + DEADLINES

↓

ONE CLAUDE NIGHTLY REPORT



Do not reverse that sequence.

Building eleven APIs before proving that the nightly report changes tomorrow's behavior would be classic engineering overkill.

The most important metric for CollegeOS itself

I would measure the system with one north-star outcome:

Did CollegeOS increase the percentage of high-value intentions that became completed actions?

Supporting metrics:

MIT completion rate
deadline lateness
deep-work completion
planned-vs-actual start delay
task-duration prediction accuracy
kill-habit frequency
bounce-back time
academic risk
weekly planning accuracy
manual-entry burden

morning check-in completion
night-review completion

And there should be a meta-rule:

If a metric does not influence a decision
for 30 days,
flag it for deletion.

The eventual value of this tool will not come from having the largest quantified-self database possible. Existing products already demonstrate that automatic tracking, guided planning, adaptive scheduling and commitment mechanisms can each be useful independently. ³⁶

Your advantage is combining them into one feedback system that knows **what college obligations you have, what you intended to do, what you actually did, what conditions surrounded the result, why you think it happened, whether that explanation survives skeptical analysis, and exactly what should change tomorrow.**

That is the version I would build.

1 ³⁶ Exist · Understand your behaviour.

https://exist.io/?utm_source=chatgpt.com

2 Implementation Intentions and Goal Achievement: A Meta- ...

https://www.sciencedirect.com/science/article/pii/S0065260106380021?utm_source=chatgpt.com

3 Momentary Factors and Study Characteristics Associated With ...

https://formative.jmir.org/2024/1/e49512?utm_source=chatgpt.com

4 Brightspace - Innovative Learning

https://www.purdue.edu/innovativelearning/tools-resources/instructional-technology/brightspace/?utm_source=chatgpt.com

5 A large-scale test of the relationship between ...

https://sure.sunderland.ac.uk/id/eprint/11813/3/A%20large-scale%20test%20of%20the%20relationship.pdf?utm_source=chatgpt.com

6 ¹⁰ WHOOP API Docs | WHOOP for Developers

https://developer.whoop.com/api/?utm_source=chatgpt.com

7 Timeboxing: the basics — Sunsama User Manual

https://help.sunsama.com/docs/getting-started/basics/timeboxing-the-basics/?utm_source=chatgpt.com

8 ²⁷ Implementation Intentions

https://cancercontrol.cancer.gov/brp/research/constructs/implementation-intentions?utm_source=chatgpt.com

9 Standard Beeminder Contract

https://www.beeminder.com/contract?utm_source=chatgpt.com

11 **Analytic Data API: What You Can Access - RescueTime**

https://help.rescuetime.com/article/465-analytic-data-api-what-you-can-access?utm_source=chatgpt.com

12 **Screen Time Technology Frameworks**

https://developer.apple.com/documentation/screentimeapidocumentation?utm_source=chatgpt.com

13 **Screen Time | Apple Developer Forums**

https://developer.apple.com/forums/tags/screen-time?page=3&utm_source=chatgpt.com

14 **HealthKit | Apple Developer Documentation**

https://developer.apple.com/documentation/healthkit?utm_source=chatgpt.com

15 **Authentication Scopes Table - Brightspace API - D2L**

https://docs.valence.desire2learn.com/http-scopestable.html?utm_source=chatgpt.com

16 **Brightspace API Authentication Guide - OAuth 2.0**

https://community.d2l.com/brightspace/kb/articles/1134-brightspace-api-authentication-guide-oauth-2-0?utm_source=chatgpt.com

17 **Manage course events with the Calendar tool - Brightspace**

https://community.d2l.com/brightspace/kb/articles/18153-manage-course-events-with-the-calendar-tool?utm_source=chatgpt.com

18 **Outlook calendar API overview - Microsoft Graph**

https://learn.microsoft.com/en-us/graph/outlook-calendar-concept-overview?utm_source=chatgpt.com

19 **Todoist API**

https://developer.todoist.com/api/v1/?utm_source=chatgpt.com

20 **Introduction - Notion Docs**

https://developers.notion.com/reference/intro?utm_source=chatgpt.com

21 **Zotero Web API Documentation**

https://www.zotero.org/support/dev/web_api/v3/basics?utm_source=chatgpt.com

22 **Readwise API**

https://readwise.io/api_deets?utm_source=chatgpt.com

23 **Models overview - Claude Platform Docs**

<https://docs.anthropic.com/en/docs/about-claude/models/overview>

24 **Choose a Claude plan | Claude Help Center**

https://support.claude.com/en/articles/11049762-choose-a-claude-plan?utm_source=chatgpt.com

25 **Overview - Claude Code Docs**

https://docs.anthropic.com/en/docs/claude-code/overview?utm_source=chatgpt.com

26 **API and data retention - Claude Platform Docs**

<https://docs.anthropic.com/en/docs/build-with-claude/zero-data-retention>

28 **What's new in Screen Time API - WWDC22 - Videos**

https://developer.apple.com/videos/play/wwdc2022/110336/?utm_source=chatgpt.com

29 **I have a paid Claude subscription (Pro, Max, Team, or Enterprise plans). Why do I have to pay separately to use the Claude API and Console? | Claude Help Center**

<https://support.claude.com/en/articles/9876003-i-have-a-paid-claude-subscription-pro-max-team-or-enterprise-plans-why-do-i-have-to-pay-separately-to-use-the-claude-api-and-console>

30 Use the Claude Agent SDK with your Claude plan | Claude Help Center

<https://support.claude.com/en/articles/15036540-use-the-claude-agent-sdk-with-your-claude-plan>

31 32 34 Pricing - Claude Platform Docs

<https://docs.anthropic.com/en/docs/about-claude/pricing>

33 Token counting - Claude Platform Docs

https://docs.anthropic.com/en/docs/build-with-claude/token-counting?utm_source=chatgpt.com

35 Hooks reference - Claude Code Docs

https://docs.anthropic.com/en/docs/claude-code/hooks?utm_source=chatgpt.com